

1

WHAT IS THE SHELL?



When we speak of the command line, we are really referring to the *shell*. The shell is a program that takes keyboard commands and passes them to the operating system to carry out. Almost all Linux distributions supply a shell program from the GNU Project called *bash*. The name is an acronym for *bourne again shell*, a reference to the fact that *bash* is an enhanced replacement for *sh*, the original Unix shell program written by Steve Bourne.

Terminal Emulators

When using a graphical user interface (GUI), we need another program called a *terminal emulator* to interact with the shell. If we look through our desktop menus, we will probably find one. KDE uses konsole, and GNOME uses gnome-terminal, though it's likely called simply “terminal” on your menu. A number of other terminal emulators are available for Linux, but they all basically do the same thing: give us access to the shell. You will probably develop a preference for one or another terminal emulator based on the number of bells and whistles it has.

Making Your First Keystrokes

So let's get started. Launch the terminal emulator. Once it comes up, we should see something like this:

```
[me@linuxbox ~]$
```

This is called a *shell prompt*, and it will appear whenever the shell is ready to accept input. While it may vary in appearance somewhat depending on the distribution, it will typically include your *username@machinename*, followed by the current working directory (more about that in a little bit) and a dollar sign.

NOTE

If the last character of the prompt is a hash mark (#) rather than a dollar sign, the terminal session has superuser privileges. This means either we are logged in as the root user or we selected a terminal emulator that provides superuser (administrative) privileges.

Assuming things are good so far, let's try some typing. Enter some gibberish at the prompt like so:

```
[me@linuxbox ~]$ kaekfjaeifj
```

Because this command makes no sense, the shell tells us so and gives us another chance:

```
bash: kaekfjaeifj: command not found  
[me@linuxbox ~]$
```

Command History

If we press the up arrow, we will see that the previous command `kaekfjaeifj` reappears after the prompt. This is called *command history*. Most Linux distributions remember the last 1,000 commands by default. Press the down arrow and the previous command disappears.

Cursor Movement

Recall the previous command by pressing the up arrow again. If we try the left and right arrows, we'll see how we can position the cursor anywhere on the command line. This makes editing commands easy.

A FEW WORDS ABOUT MICE AND FOCUS

While the shell is all about the keyboard, you can also use a mouse with your terminal emulator. A mechanism built into the windowing system (the underlying engine that makes the GUI go) supports a quick copy-and-paste technique. If you highlight some text by holding down the left mouse button and dragging the mouse over it (or double-clicking a word), it is copied into a buffer maintained by the windowing system. Pressing the middle mouse button will cause the text to be pasted at the cursor location. Try it. It works on most desktop environments.

Don't be tempted to use `CTRL-C` and `CTRL-V` to copy and paste inside a terminal window. They don't work. These control codes have different meanings to the shell and were assigned many years before the release of Microsoft Windows. Use `SHIFT-CTRL-C` and `SHIFT-CTRL-V` instead while using the terminal window.

Your graphical desktop environment (most likely KDE or GNOME), in an effort to behave like Windows, probably has its focus policy set to "click to focus." This means for a window to get focus (become active), you need to click it. This is contrary to the traditional window system behavior of "focus follows mouse," which means that a window gets focus just by passing the mouse over it. The window will not come to the foreground until you click it, but it will be able to receive input. Setting the focus policy to "focus follows mouse" will make the copy-and-paste technique even more useful. Try it if you can (though some desktop environments no longer support it). I think if you give it a chance, you will prefer it. You will find this setting in the configuration program for your window manager.

Try Some Simple Commands

Now that we have learned to enter text in our terminal emulator, let's try a few simple commands. Let's begin with the `date` command, which displays the current time and date:

```
[me@linuxbox ~]$ date  
Thu Mar  8 15:09:41 EST 2025
```

Another handy command is `uptime`, which displays how long the system has been running and the average number of processes running over various periods of time:

```
[me@linuxbox ~]$ uptime
15:12:22 up 3 days, 23:40,  7 users,  load average: 0.37, 0.37, 0.64
```

THE CONSOLE BEHIND THE CURTAIN

Even if we have no terminal emulator running, several terminal sessions continue to run behind the graphical desktop. We can access these sessions, called *virtual terminals* or *virtual consoles*, by pressing `CTRL-ALT-F1` through `CTRL-ALT-F6` on most Linux distributions. When a session is accessed, it presents a login prompt into which we can enter our username and password. To switch from one virtual console to another, press `ALT-F1` through `ALT-F6`. On most systems, we can return to the graphical desktop by pressing `ALT-F7`.

To see the current amount of free space on our disk drives, enter `df`:

```
[me@linuxbox ~]$ df
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/sda2	15115452	5012392	9949716	34%	/
/dev/sda5	59631908	26545424	30008432	47%	/home
/dev/sda1	147764	17370	122765	13%	/boot
tmpfs	256856	0	256856	0%	/dev/shm

Likewise, to display the amount of free memory, enter the `free` command:

```
[me@linuxbox ~]$ free
```

	total	used	free	shared	buffers	cached
Mem:	513712	503976	9736	0	5312	122916
-/+ buffers/cache:	375748	137964				
Swap:	1052248	104712	947536			

Ending a Terminal Session

We can end a terminal session by either closing the terminal emulator window, entering the `exit` command at the shell prompt, or pressing `CTRL-D`.

```
[me@linuxbox ~]$ exit
```

Summing Up

This chapter marked the beginning of our journey into the Linux command line with an introduction to the shell, a brief glimpse of the command line, and a lesson on how to start and end a terminal session. We also saw how to issue some simple commands and perform a little light command line editing. That wasn't so scary, was it?

In the next chapter, we'll learn a few more commands and wander around the Linux filesystem.